

DOCUMENTO SAD – EcoRoute

1. INTRODUCCIÓN

1.1 Propósito del Documento

Este documento describe la arquitectura de software del sistema EcoRoute, detallando las decisiones estructurales, los componentes del sistema, sus responsabilidades, las relaciones entre ellos y los patrones arquitectónicos aplicados. Está dirigido al equipo de desarrollo e investigadores que necesiten comprender cómo está organizado internamente el sistema para extenderlo, mantenerlo o replicarlo.

1.2 Alcance

El SAD cubre la arquitectura del sistema EcoRoute en su versión inicial: control de una intersección urbana aislada mediante un agente DQN entrenado sobre el simulador SUMO. No cubre la extensión futura a arquitecturas multi-agente ni la integración con infraestructura vial física.

2. REPRESENTACIÓN ARQUITECTÓNICA

2.1 Estilo Arquitectónico

EcoRoute sigue un estilo de arquitectura por capas con pipeline de datos, complementado con el patrón Environment-Agent estándar en sistemas de aprendizaje por refuerzo. Cada capa tiene una responsabilidad única y se comunica únicamente con las capas adyacentes, lo que garantiza bajo acoplamiento y alta cohesión entre módulos.

2.2 Vistas Arquitectónicas Incluidas

Vista	Descripción
Vista de componentes	Módulos del sistema y sus responsabilidades
Vista de conectores	Flujo de datos e interacciones entre componentes
Vista de despliegue	Entorno de ejecución y dependencias de infraestructura
Vista de capas	Organización jerárquica de los módulos

3. METAS Y RESTRICCIONES ARQUITECTÓNICAS

Código	Meta / Restricción	Impacto en la arquitectura
MA-01	Modularidad: cada módulo debe poder modificarse de forma independiente	Separación estricta entre entorno, agente, recompensa y evaluación
MA-02	Reproducibilidad: resultados idénticos con la misma semilla	Control de semillas centralizado en el módulo de configuración

MA-03	Compatibilidad Gym: interfaz del entorno compatible con OpenAI/Gymnasium	El módulo de entorno implementa la interfaz reset(), step(), render()
MA-04	Escalabilidad futura a multi-agente	Agente y entorno diseñados como clases instanciables e independientes
MA-05	Portabilidad Linux/Windows	Sin dependencias de sistema operativo; uso de rutas relativas y conda/pip
MA-06	Seguridad vial en simulación	Validación de acciones dentro del módulo de entorno antes de enviar a SUMO

4. VISTA DE COMPONENTES

4.1 Descripción de Componentes

Campo	Detalle
Componente	ENV — Módulo de Entorno
Capa	Capa 1
Archivo / Clase	environment/sumo_env.py · Clase SumoEnv
Responsabilidad	Encapsular la interacción con SUMO vía TraCI; construir el vector de estado; validar y ejecutar acciones semafóricas; devolver estado, recompensa y flag de fin de episodio
Interfaz expuesta	reset() → state, step(action) → (state, reward, done, info), render()
Depende de	SUMO (externo), TraCI (biblioteca Python), Módulo de Emisiones, Módulo de Recompensa
Restricciones	Debe implementar la interfaz Gym; no puede ejecutar acciones que violen restricciones de seguridad vial

Campo	Detalle
Componente	EMI — Módulo de Emisiones
Capa	Capa 2
Archivo / Clase	environment/emissions.py · Clase EmissionCalculator

Responsabilidad	Calcular las emisiones de CO ₂ en gramos por paso de simulación usando el modelo VSP o COPERT simplificado, a partir de velocidad y aceleración vehicular
Interfaz expuesta	calculate(speed, acceleration, vehicle_type) → co2_grams
Depende de	Datos de velocidad y aceleración provenientes de SUMO vía TraCI
Restricciones	Debe ser intercambiable entre modelo VSP y COPERT sin modificar el módulo de entorno

Campo	Detalle
Componente	REW — Módulo de Recompensa
Capa	Capa 3
Archivo / Clase	environment/reward.py · Clase RewardCalculator
Responsabilidad	Calcular el valor escalar de recompensa combinando penalización por tiempo de espera, penalización por emisiones y bonificación por throughput, con pesos α , β , γ configurables
Interfaz expuesta	compute(waiting_time, co2_emissions, throughput) → reward_scalar
Depende de	Módulo de Emisiones (valor de CO ₂), datos de espera y throughput desde SUMO
Restricciones	Los pesos α , β , γ deben ser configurables vía archivo de configuración sin modificar el código fuente

Campo	Detalle
Componente	AGT — Módulo del Agente DQN
Capa	Capa 4
Archivo / Clase	agent/dqn_agent.py · Clase DQNAgent
Responsabilidad	Mantener la red neuronal principal y la target network; gestionar el replay buffer; implementar la política ϵ -greedy; calcular la pérdida TD y actualizar los pesos mediante backpropagation
Interfaz expuesta	select_action(state) → action, store_transition(s, a, r, s', done), train_step() → loss, update_target_network()

Depende de	PyTorch (red neuronal y optimizador), Módulo de Replay Buffer
Restricciones	La target network debe actualizarse solo cada N pasos (configurable); el replay buffer no inicia entrenamiento hasta alcanzar un tamaño mínimo configurable

Campo	Detalle
Componente	BUF — Replay Buffer
Capa	Capa 4 (sub-componente del agente)
Archivo / Clase	agent/replay_buffer.py · Clase ReplayBuffer
Responsabilidad	Almacenar transiciones (s, a, r, s', done) con capacidad máxima configurable; proveer muestras aleatorias de tamaño batch_size para el entrenamiento
Interfaz expuesta	push(transition), sample(batch_size) → batch, __len__() → int
Depende de	Ningún otro módulo interno
Restricciones	Implementar como buffer circular (FIFO) para respetar la capacidad máxima sin crecimiento ilimitado de memoria

Campo	Detalle
Componente	TRN — Módulo de Entrenamiento
Capa	Capa 5
Archivo / Clase	training/trainer.py · Clase Trainer
Responsabilidad	Orquestar el loop principal de entrenamiento por episodios; gestionar la interacción entre el agente y el entorno; guardar checkpoints del modelo; controlar el decaimiento de ϵ a lo largo del entrenamiento
Interfaz expuesta	train(n_episodes) → metrics, evaluate(n_episodes) → metrics, save_checkpoint(path), load_checkpoint(path)
Depende de	Módulo de Entorno, Módulo del Agente DQN, Módulo de Evaluación
Restricciones	Debe controlar la semilla aleatoria global al inicio de cada experimento

Campo	Detalle
Componente	EVA — Módulo de Evaluación y Visualización
Capa	Capa 6
Archivo / Clase	evaluation/logger.py · evaluation/visualizer.py
Responsabilidad	Registrar métricas por episodio (recompensa acumulada, emisiones, tiempo de espera, throughput, pérdida); exportar datos a CSV; generar gráficas de convergencia, mapas de calor de densidad vehicular y comparativas de emisiones entre políticas
Interfaz expuesta	log_episode(metrics_dict), export_csv(path), plot_convergence(), plot_emissions_comparison(), plot_heatmap()
Depende de	matplotlib, seaborn, pandas
Restricciones	Las gráficas deben exportarse en PNG y PDF; el CSV debe incluir encabezados descriptivos

Campo	Detalle
Componente	CFG — Módulo de Configuración
Capa	Transversal (usada por todas las capas)
Archivo / Clase	config/config.yaml · config/config_loader.py
Responsabilidad	Centralizar todos los parámetros configurables del sistema: hiperparámetros del agente, pesos de recompensa, parámetros de simulación, semillas, rutas de archivos y modelo de emisiones a usar
Interfaz expuesta	load_config(path) → dict
Depende de	PyYAML
Restricciones	Ningún módulo debe tener parámetros hardcodeados; todo valor configurable debe provenir del archivo YAML

5. VISTA DE CONECTORES — FLUJO DE DATOS

5.1 Flujo Principal: Paso de Simulación

Paso	Origen	Destino	Dato transferido
------	--------	---------	------------------

1	TRN (Trainer)	ENV (SumoEnv)	Llamada a step(action)
2	ENV	SUMO (externo)	Acción semafórica vía TraCI
3	SUMO	ENV	Estado del tráfico (densidad, cola, fase, velocidad, aceleración)
4	ENV	EMI	Velocidad y aceleración por vehículo
5	EMI	REW	Emisiones de CO ₂ en gramos
6	ENV	REW	Tiempo de espera acumulado y throughput
7	REW	ENV	Valor escalar de recompensa
8	ENV	TRN	Tupla (state, reward, done, info)
9	TRN	AGT	store_transition(s, a, r, s', done)
10	TRN	AGT	train_step()
11	AGT	BUF	sample(batch_size)
12	TRN	EVA	Métricas del paso/episodio

5.2 Flujo de Selección de Acción

Paso	Origen	Destino	Dato transferido
1	TRN	AGT	select_action(state)
2	AGT	AGT	Política ϵ -greedy: aleatorio vs. argmax Q(s,a)
3	AGT	TRN	Acción seleccionada (0 = mantener, 1 = cambiar fase)
4	TRN	ENV	Acción para ejecutar en el siguiente paso

5.3 Flujo de Actualización de la Red

Paso	Origen	Destino	Dato transferido
1	AGT	BUF	sample(batch_size) → lote de transiciones
2	AGT	Red principal	Forward pass con estados del lote
3	AGT	Target Network	Forward pass con estados siguientes del lote
4	AGT	AGT	Cálculo de pérdida TD: $L = (r + \gamma \cdot \max Q'(s') - Q(s,a))^2$
5	AGT	Red principal	Backpropagation y actualización de pesos

6	TRN	AGT	update_target_network() cada N pasos
---	-----	-----	--------------------------------------

6. VISTA DE DESPLIEGUE

6.1 Dependencias de Software

Dependencia	Versión recomendada	Rol en el sistema
Python	3.10 o superior	Lenguaje base del sistema
SUMO	1.18 o superior	Simulador de tráfico urbano
TraCI	Incluido con SUMO	Interfaz de control de SUMO desde Python
PyTorch	2.0 o superior	Framework de red neuronal del agente DQN
NumPy	1.24 o superior	Operaciones vectoriales sobre estados y lotes
pandas	2.0 o superior	Registro y exportación de métricas a CSV
matplotlib	3.7 o superior	Generación de gráficas de convergencia y comparativas
seaborn	0.12 o superior	Mapas de calor de densidad vehicular
PyYAML	6.0 o superior	Carga del archivo de configuración
Gymnasium	0.29 o superior	Interfaz estándar del entorno de RL

7. DECISIONES ARQUITECTÓNICAS CLAVE

Decisión	Impacto en la arquitectura
Uso de SUMO como simulador	El módulo de entorno queda acoplado a la API TraCI; se aísla este acoplamiento dentro de sumo_env.py para que el resto del sistema no dependa directamente de SUMO
Algoritmo DQN con Experience Replay	Requiere el componente BUF (Replay Buffer) como sub-módulo del agente; impone asincronía entre recolección de experiencia y actualización de pesos
Función de recompensa híbrida escalar	Permite un agente único en lugar de optimización multi-objetivo; los pesos α , β , γ quedan en el módulo CFG para ajuste experimental

Interfaz Gym-compatible	El módulo ENV debe implementar reset() y step() siguiendo el contrato Gym, lo que permite futura integración con bibliotecas como Stable-Baselines3
Alcance de una intersección aislada	El módulo ENV instancia un solo entorno SUMO; la extensión multi-agente requeriría instanciar múltiples ENV y AGT coordinados, pero la estructura actual lo permite sin rediseño
Configuración centralizada en YAML	Ningún módulo tiene parámetros hardcodeados; todos leen desde CFG, lo que facilita la reproducibilidad y la ejecución de barridos de hiperparámetros

8. ATRIBUTOS DE CALIDAD Y CÓMO LA ARQUITECTURA LOS SATISFACE

Atributo	Mecanismo arquitectónico que lo satisface
Modularidad	Separación estricta en 6 capas con interfaces bien definidas; cada módulo tiene una única responsabilidad
Reproducibilidad	Control de semilla centralizado en TRN al inicio de cada experimento; configuración en YAML versionable
Escalabilidad	Agente y entorno como clases instanciables; arquitectura preparada para instancias múltiples en extensión multi-agente
Mantenibilidad	Interfaz Gym-compatible permite reemplazar SUMO por otro simulador modificando solo sumo_env.py; modelo de emisiones intercambiable en emissions.py
Seguridad vial	Validación de acciones dentro de ENV antes de enviarlas a SUMO; el agente nunca interactúa directamente con el simulador
Portabilidad	Sin dependencias de sistema operativo; entorno conda/pip documentado; rutas relativas en toda la estructura
Validez científica	Módulo EVA registra todas las métricas con precisión por episodio y corrida; soporte para múltiples semillas desde TRN